

Security Findings Report

Vulnerability Assessment of the **iot-motor-api-vulnerable** sample application

Project	IBM Bob 2.0 Hackathon — Team Othy
Component	iot-motor-api-vulnerable (Node.js / Express IoT motor-config API)
Purpose	Vulnerable-by-design training target; input to the automated CVE-remediation pipeline
Prepared by	Kent Vincent S. Godinez — Role 4 (IoT Backend Developer)
Date	September 27, 2026
Classification	Internal — educational / security-training use only

Notice. This application was written intentionally insecure for security education and to serve as a controlled test target. Every weakness described here is deliberate. Do not deploy this software, expose it to a network, or reuse its patterns. This report documents the flaws so the remediation pipeline (and human reviewers) can verify each fix.

1. Executive Summary

The **iot-motor-api-vulnerable** service is a small Express backend that accepts JSON configuration commands for a fleet of TB6612FNG-driven 6V N20 micro gear motors (PWM duty cycle, drive frequency, oscillator clock, and direction). This assessment identified **six** distinct weaknesses spanning input handling, dependency hygiene, information disclosure, and transport hardening.

The most serious is a **prototype-pollution** vulnerability (F-01): the configuration endpoint deep-merges untrusted request bodies into application state using an outdated **lodash** version whose merge routine walks `__proto__` keys. A single crafted request can modify `Object.prototype` for the entire Node.js process, corrupting unrelated logic and enabling denial of service or logic bypass. Because the same endpoint performs no input validation (F-02), a caller can also drive motor parameters far outside the hardware-safe envelope — a physical-safety concern for an IoT control plane.

Severity	Count	Findings
Critical	1	F-01
High	2	F-02, F-03
Medium	3	F-04, F-05, F-06

2. Scope & Methodology

Scope. Static source review and dynamic testing of the single-service codebase: `src/server.js`, `src/api/n20-config.js`, and the declared dependencies in `package.json`.

Method. Manual code review; dependency analysis via `npm audit`; and dynamic confirmation by sending probe requests to a locally-run instance. Each finding below was reproduced against a running build — see `tests/smoke.test.js`, which asserts that F-01, F-02, and F-04 are live on the current (vulnerable) code.

Remediation target. The intended secure end-state is defined in `docs/hardware-sec-memo.md`. A fix is complete when every finding is closed and the smoke assertions for F-01/F-02/F-04 invert (pollution blocked, out-of-range values rejected with HTTP 400, no stack trace returned).

3. Detailed Findings

F-01 Prototype Pollution via lodash.merge on the config endpoint				CRITICAL
CWE	CWE-1321 (Improperly Controlled Modification of Object Prototype Attributes)	Location	src/api/n20-config.js — POST /api/motor/config	
CVE / Adv.	CVE-2018-3721, CVE-2018-16487	Status	Open (by design)	

Description. The handler calls `_.merge(motorState, req.body)`, deep-merging the untrusted request body directly into shared application state. The pinned dependency `lodash@4.17.4` predates the merge prototype-pollution fixes (4.17.5 / 4.17.11), so its recursive merge follows a `__proto__` key in the payload and assigns onto the global object prototype rather than the target object.

Illustration

```
POST /api/motor/config Content-Type: application/json
{ "__proto__": { "polluted": "yes" } }
```

After this request, `({}).polluted === "yes"` for every object in the process.

Impact. Process-wide corruption of `Object.prototype`. Depending on the injected key, this can cause denial of service (crashes / unexpected branches), bypass of logic that reads object properties (e.g. flags controlling motor limits), or serve as a stepping stone to more severe impact. It affects all requests handled by the process, not just the attacker's.

Recommended remediation. Stop deep-merging untrusted input. Validate the body against a strict field allowlist and build a fresh state object from only known keys. Reject payloads containing `__proto__`, `constructor`, or `prototype`. Upgrade `lodash` to a current release. This is the change the hardware-sec-memo mandates (route parsing through a safe configurator).

F-02 Missing input validation — unsafe motor parameters accepted				HIGH
CWE	CWE-20 (Improper Input Validation)	Location	src/api/n20-config.js — POST /api/motor/config	
CVE / Adv.	—	Status	Open (by design)	

Description. No validation is performed on incoming fields. Arbitrary/unknown keys are stored, and known fields accept values well outside the hardware-safe operating envelope defined in the memo (e.g. `pwmDutyMax` 0–100%, `clockHz` 1–16 MHz). A request setting `pwmDutyMax: 250` or `clockHz: 99999999` is accepted and persisted.

Illustration

```
POST /api/motor/config
{ "pwmDutyMax": 250, "clockHz": 99999999 } → 200 OK (stored)
```

Impact. For a control plane that drives physical actuators, accepting out-of-range duty cycle or clock values is a safety and reliability risk (motor overdrive, overheating, undefined hardware behaviour), and it compounds F-01 by giving an attacker a second way to reach unexpected state.

Recommended remediation. Enforce a strict schema: allowlist field names, type-check values, and range-check `pwmDutyMax`, `pwmFreqHz`, `clockHz`, `direction`, and `motorId`. Reject the whole payload with

HTTP 400 on any violation (all-or-nothing; no partial application).

F-03 Outdated dependency with multiple known CVEs (lodash@4.17.4)			HIGH
CWE	CWE-1104 (Use of Unmaintained / Vulnerable Third-Party Components)	Location	package.json — dependencies.lodash
CVE / Adv.	CVE-2018-3721, CVE-2018-16487, CVE-2019-10744, CVE-2020-8203, CVE-2021-23337	Status	Open (by design)

Description. The project pins `lodash@4.17.4`, which is several years out of date. `npm audit` reports numerous advisories against this version, including the merge prototype-pollution issues behind F-01, additional prototype-pollution in `defaultsDeep/zipObjectDeep`, command injection via `_.template`, and ReDoS.

Illustration

```
Run: npm audit (reports lodash advisories, high & critical severity)
```

Impact. Even code paths the app does not currently use represent latent risk if the dependency surface changes, and the version directly enables F-01.

Recommended remediation. Upgrade `lodash` to the latest 4.17.x (or remove the dependency entirely, since a validated allowlist parser needs no deep-merge library). Add `npm audit` to CI so regressions are caught automatically.

F-04 Information disclosure — stack traces returned to clients			MEDIUM
CWE	CWE-209 (Generation of Error Message Containing Sensitive Information)	Location	src/server.js — error-handling middleware
CVE / Adv.	—	Status	Open (by design)

Description. The global error handler returns `err.message` and `err.stack` in the JSON response body, exposing internal file paths, dependency versions, and code structure to any caller who can trigger an error.

Illustration

```
500 response body: { "error": "...", "stack": "Error: ...\n at /app/src/..." }
```

Impact. Leaked internals accelerate reconnaissance and help an attacker tailor further attacks (including against F-01/F-03).

Recommended remediation. Return a generic error message and a correlation ID to the client; log full detail server-side only. Never include `err.stack` in a response.

F-05 Missing HTTP hardening — no security headers, no rate limiting			MEDIUM
CWE	CWE-693 (Protection Mechanism Failure)	Location	src/server.js — middleware stack
CVE / Adv.	—	Status	Open (by design)

Description. No security-header middleware (e.g. `helmet`) is installed, so responses omit headers such as `X-Content-Type-Options` and `Content-Security-Policy`. There is also no rate limiting, so the config endpoint can be flooded unthrottled.

Impact. Weaker defense-in-depth and exposure to brute-force / flooding against the config endpoint, which amplifies F-01 (each request can attempt pollution).

Recommended remediation. Add `helmet()` for baseline security headers, apply a request rate limiter to the API routes, and set a sensible JSON body-size limit on `express.json()`.

F-06 Unstructured logging of full request bodies			MEDIUM
CWE	CWE-532 (Insertion of Sensitive Information into Log File)	Location	src/api/n20-config.js and src/server.js — <code>console.log</code> calls
CVE / Adv.	—	Status	Open (by design)

Description. Incoming request bodies and raw error objects are written verbatim with `console.log`, with no structure, redaction, or level control. The memo explicitly bans raw `console.log` for configuration and 4 MHz oscillator-sync events.

Impact. Verbatim logging can capture unexpected or sensitive payload contents and produces logs that are hard to monitor or filter; it also violates the internal governance mandate.

Recommended remediation. Introduce a structured, level-aware logger with redaction of unknown/sensitive fields; log a summary rather than the full body. Provide a silent mode for tests.

4. Remediation Priority Matrix

ID	Finding	Severity	Fix effort	Priority
F-01	Prototype pollution (lodash.merge)	Critical	Medium	1
F-03	Outdated lodash (known CVEs)	High	Low	2
F-02	No input validation	High	Medium	3
F-04	Stack-trace disclosure	Medium	Low	4
F-05	Missing HTTP hardening	Medium	Low	5
F-06	Unstructured logging	Medium	Low	6

Recommended order: close F-01 and F-03 together (upgrade lodash and replace the deep-merge with an allowlist parser), which also delivers F-02; then F-04, F-05, and F-06 as straightforward hardening. A remediation is verified when the smoke assertions for F-01/F-02/F-04 invert on the patched build.

iot-motor-api-vulnerable — Security Findings Report · Team Othy (Role 4) · For educational / security-training use only.